

T1 – Paralelização de uma aplicação com OpenMP

Computação Paralela – Unidade 03 | Aluno: Bernardo Klein Heitz

Código entregue em .zip junto deste PDF.

1. O problema e o potencial de paralelização

Paralelizamos com OpenMP um *path tracer* no estilo *smallpt* de Beason [1], reescrito em C11 (padrão ISO C de 2011). O programa integra a iluminância de uma Cornell Box de nove esferas (paredes difusas, uma especular e uma refrativa) por amostragem de Monte Carlo, com 2×2 subpixels e roleta russa a partir da profundidade 5. A entrada `-w 800 -h 600 -s 32` coloca o sequencial em $T_1 \approx 54$ s: há um laço externo independente por linha, os materiais impedem um `parallel for` trivial, e w , h e spp parametrizam o trabalho.

A escolha do eixo veio do *gprof* na mesma entrada e na mesma fase do speed-up (binário `-O2 -fno-inline -pg`). *radiance* e as rotinas que ela chama somam 98,2% do tempo; *main* fica com 1,75%. A fração paralelizável é $f \approx 0,982$, logo $S_{\max}(6) = 1/(0,018 + 0,982/6) \approx 5,51$, $S_{\max}(12) \approx 10,00$ e $S_{\infty} = 1/(1 - f) \approx 55,6$. No binário `-O3` de medição, $\text{time_par_s}/\text{time_s} = 0,99998$: a parcela serial da fase medida é desprezível. O $7,44\times$ da Seção 5 fica abaixo de $S_{\max}(12)$, portanto a saturação não se deve a essa fração.

2. Ambiente e metodologia

CPU AMD Ryzen 5 3600 (6 núcleos físicos, 12 processadores lógicos, 3,6 GHz base), L3 de 32 MB, 32 GB de RAM DDR4. SO Ubuntu 22.04 em WSL2 sobre Windows 11. Compilador gcc 11.4.0, flags idênticas `-O3 -march=native -ffast-math -std=c11`; a versão paralela acrescenta só `-fopenmp`. T_1 vem do binário compilado sem `-fopenmp`, não da versão OpenMP com uma thread. `omp_get_num_procs()` devolve 12. Medimos com `omp_get_wtime()` a fase computacional completa (câmera, alocação, laço e checksum); escrita PPM e print CSV ficam fora, nos dois binários. Cada ponto teve 10 execuções em máquina ociosa; reportamos mediana, mínimo e máximo. Threads fixadas por `OMP_NUM_THREADS`. O checksum RGB é idêntico de 1 a 12 threads e em todas as políticas.

3. Modelagem e diretivas

A unidade de trabalho é uma linha da imagem (600 iterações em y). As linhas não são homogêneas: as que cruzam o espelho e o vidro ricocheteiam mais; as do fundo preto terminam cedo. O perfil com `nowait` mesmo assim mostra desvio entre threads abaixo de 1%; o que pesa é a preemp-

ção do WSL2, não a cena. Cada iteração escreve numa faixa disjunta de c , o que dispensa `atomic`, `critical` e `reduction` no caminho quente. Um gerador global exigiria `critical` em cada `rng_next` e serializaria dezenas de milhões de chamadas. O `xorshift64` ficou local ao corpo do laço, com semente derivada de y , e o checksum deixou de depender do número de threads.

Os pragmas estão em `src/smallpt.c:320-332`: `#pragma omp parallel default(none) shared(...)` envolve `#pragma omp for schedule(runtime) nowait` sobre o laço em y . O `default(none)` obriga listar o escopo. O buffer c tem escrita disjunta; `stat` é indexado por `omp_get_thread_num()` com padding de 64 B contra falso compartilhamento. `schedule(runtime)` troca a política por `OMP_SCHEDULE`; `nowait` faz o tempo por thread refletir a carga real, não a barreira.

4. Balanceamento de carga

Testamos sete políticas em 12 threads, 10 execuções cada (Tabela 3). `dynamic,4` teve a menor mediana (7,16s) e a menor dispersão (7,01 a 7,35s). `static` sem chunk ficou em último (9,56s; 7,57 a 11,78s), embora o desvio interno seja inferior a 1%. A heterogeneidade das linhas não explica esse gap: no WSL2 uma thread preemptada no `static` segura o bloco inteiro, enquanto `dynamic,4` redistribui pedaços de quatro linhas. Adotamos `dynamic,4` por essa medição, nas duas escalabilidades.

5. Análise dos resultados

A Figura 1 e a Tabela 1 descrevem a escalabilidade forte (entrada fixa). Até 6 threads o speed-up acompanha Amdahl: 4,93 medidos contra o teto 5,51, eficiência 82,2%. O afastamento já visível em 4 threads (3,31 contra o teto 3,73) não é fração sequencial – essa parcela é $< 0,002\%$ da fase medida –, e sim o custo de criar o time OpenMP e a disputa por L3. No ponto em que o SMT entra (linha vertical em 6 threads nas Figuras 1 e 2) a curva se dobra. De 6 para 8 o speed-up vai a 6,14 (+25%) e a eficiência cai a 76,8%; de 10 para 12 o ganho adicional é 1,8% e a eficiência cai a 62,0%. O *smallpt* é limitado por ponto flutuante em `double`: duas threads no mesmo núcleo competem pelas unidades FP e pela cache. O $7,44\times$ em 12 threads fica abaixo de $S_{\max}(12) \approx 10$, logo a saturação é SMT somado à preempção do WSL2, não à fração sequencial.

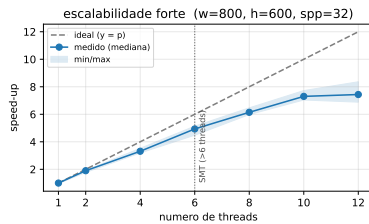


Figura 1: Speed-up da escalabilidade forte (mediana de 10 execuções) contra $y = p$. A faixa é o min-max. A linha em 6 threads marca o início do SMT.

A escalabilidade fraca (Figura 2, Tabela 2) parte da mesma entrada e escala $spp = 32 \cdot p$, porque o custo é $O(w \cdot h \cdot spp)$: o trabalho total cresce com p e o trabalho por thread permanece aproximadamente constante. Até 4 threads o tempo quase não sobe (55,6, 56,4 e 58,6s; eficiência $\geq 95\%$), o que é o comportamento esperado de uma aplicação sem fração serial visível. Em 6 threads vai a 67,7s (82,2%, igual à da forte): o primeiro núcleo extra ainda cabe na topologia

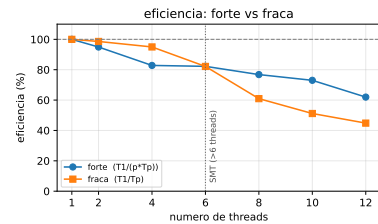


Figura 2: Eficiência forte ($T_1/(p \cdot T_p)$) e fraca (T_1/T_p). Ambas quebram no SMT; a fraca cai mais a partir de 8 threads.

física. Passado o SMT a fraca piora mais: 91,3s em 8 threads e 124,1s em 12 (44,8% contra 62,0% da forte). Com trabalho por thread constante, 12 threads em 6 núcleos colocam duas cargas completas no mesmo núcleo; o tempo esperado seria $2 \times 67,7 \approx 135$ s e medimos 124s – o SMT recupera cerca de 8%. Na forte o trabalho total é fixo, então o SMT ainda reduz 10,9s para 7,3s. As duas curvas quebram no mesmo ponto: a causa é a topologia da máquina, não o algoritmo nem Amdahl.

A dispersão das 10 repetições aponta para o mesmo sítio. Na forte, o intervalo min–max cresce de 0,3s em 2 threads para 1,4s em 12. Na fraca explode em 6 e 8 threads (60,6 a 73,5s e 76,4 a 102,6s) e fecha de novo em 12 (123,3 a 125,6s). Corridas longas no WSL2 convivem com o host Windows: o `dynamic,4` mascara a thread preemptada, mas o tempo de parede sobe. Em 12 threads a máquina já está saturada, a interferência relativa cai e a dispersão diminui. O T_1 da fraca (55,6s) difere do da forte (54,0s) porque as campanhas foram em momentos distintos; usamos o T_1 de cada campanha. A diferença de 3% é ruído de máquina. Em resumo, até os 6 núcleos físicos o path tracer escala perto do teto de Amdahl; além disso o SMT da microarquitetura Zen 2 não oferece unidades FP extras, e a fraca – que mantém trabalho por thread – denuncia isso com mais clareza que a forte.

Um assistente de IA revisou o arnês de medição, o script de plotagem e a estrutura deste relatório. Cada decisão de código (escopo, `dynamic,4`, `rng` por linha, `nowait`) foi conferida com as medições acima.

Referências

[1] K. Beason. *smallpt: global illumination in 99 lines of C++*, 2008.

Tabela 1: Escalabilidade forte ($w=800$, $h=600$, $spp=32$), 10 execuções. T_1 é o sequencial sem `-fopenmp`.

Threads	Med. (s)	Min (s)	Max (s)	Speed-up	Ef.
1 (T_1)	53,996	53,552	56,206	1,000	100,0%
2	28,437	28,126	30,418	1,899	94,9%
4	16,300	15,307	16,929	3,313	82,8%
6	10,945	10,512	12,057	4,933	82,2%
8	8,789	8,349	9,032	6,144	76,8%
10	7,395	6,991	7,672	7,302	73,0%
12	7,261	6,456	7,846	7,437	62,0%

Tabela 2: Escalabilidade fraca. Regra $spp = 32 \cdot p$ (custo linear em spp). Eficiência = T_1/T_p .

Threads	spp	Med. (s)	Min (s)	Max (s)	Ef.
1 (T_1)	32	55,643	54,551	57,554	100,0%
2	64	56,432	55,605	59,526	98,6%
4	128	58,578	57,927	59,572	95,0%
6	192	67,730	60,617	73,468	82,2%
8	256	91,290	76,362	102,594	61,0%
10	320	108,754	107,091	110,528	51,2%
12	384	124,098	123,328	125,584	44,8%

Tabela 3: Políticas de escalonamento em 12 threads, 10 execuções, mesma entrada da forte.

Política	Mediana (s)	Min (s)	Max (s)
static	9,556	7,573	11,779
static, 32	8,389	7,425	10,494
static, 4	7,658	7,259	8,287
dynamic	7,297	7,210	7,933
dynamic, 32	7,199	7,157	7,608
dynamic, 4 (adotada)	7,164	7,006	7,354
guided	7,231	6,935	7,385